



TRANSAKCJE

→ atomowa akcja (*atomic action*) = zbiór niepodzielnych operacji

Problemy

1. Współbieżność

- jak koordynować żądania współbieżnych atomowych akcji dotyczące operacji na tych samych obiektach?

2. Izolacja:

- wykonanie atomowej akcji musi być niezależne od innych atomowych akcji
- tym bardziej odtwarzanie atomowej akcji musi być niezależne od innych atomowych akcji



Konflikty żądań

Współbieżne żądania są w konflikcie jeśli dotyczą tego samego obiektu i przynajmniej jedno z nich wymaga jego modyfikacji.

- blokowanie dostępu (*locking*)

Gdy **zarządca obiektu** otrzymuje żądanie będące w konflikcie z inną akcją już wykonywaną, może w celu rozwiązywania konfliktu zastosować jedną z następujących operacji:

WAIT – nowe żądanie jest kolejkowane

REJECT – nowe żądanie jest odrzucane

PREEMPT – żądanie bieżąco obsługiwane jest anulowane

W przypadku operacji REJECT i PREEMPT odpowiednie atomowe akcje są wycofywane (*abort*). W przypadku operacji WAIT atomowa akcja może być wycofana później.



BLOKADY

2PL = *2 Phase Locking* – uszeregowanie zbioru atomowych akcji

Problem

- blokada nie zawiera sama w sobie żadnych informacji pomocnych w detekcji i / lub rozwiązywaniu konfliktów

Zakleszczenie!

- jeśli atomowa akcja oczekująca na założenie kolejnej blokady może utrzymywać wcześniej uzyskane blokady, może wystąpić zakleszczenie

Co wówczas?

- 1) detekcja zakleszczenia i abort jednej lub kilku atomowych akcji
 - 2) timeout związany z blokadą:
 - a) na utrzymywanie blokady
 - b) na oczekiwanie na założenie blokady
- ☹ timeout może prowadzić do **zbędnego** wycofywania atomowych akcji!



Transakcje rozproszone

- protokół 2PL gwarantuje uszeregowanie atomowych akcji poprzez dynamiczne uporządkowanie operacji
- alternatywnie, porządek ten może być determinowany *a priori*, poprzez globalnie jednoznaczne uszeregowanie żądań
- w systemie rozproszonym globalne uszeregowanie zapewnia mechanizm **etykiet czasowych** (*timestamps*):

globalna etykieta czasowa $TS = \langle clock(i), unique_node_ID(i) \rangle$



Strategie rozstrzygania konfliktów

- 1) zarządca obiektów zakłada blokady zgodnie z uszeregowaniem żądań
- 2) rozstrzyganie konfliktów zakładania blokad:
 - **WAIT-DIE**: nowe żądanie i czeka na blokadę posiadaną przez j jeśli $TS_i < TS_j$;
natomiast $TS_i \geq TS_j \Rightarrow i$ jest odrzucane (akcja jest wycofywana)
 - **WOUND-WAIT**: $TS_i < TS_j \Rightarrow j$ jest odrzucane; $TS_i \geq TS_j \Rightarrow i$ czeka

Problem: zbędne wycofywanie akcji (por. timeout); niezawodność zegarów



ATOMOWE ZATWIERDZANIE

Problem:

- Należy zagwarantować globalną atomowość rozproszonej transakcji:

Zgodność: *transakcję lokalnie zatwierdzają wszystkie węzły, albo żaden*

Poprawność: *jeśli wszystkie węzły zakończyły swoje operacje pomyślnie, transakcja powinna zostać zatwierdzona*

Protokół zatwierdzania (**commitment**):

- Jego rolą jest zagwarantowanie globalnej atomowości (i trwałości).
- Każda składowa transakcja lokalnie dokonuje modyfikacji zapisując rejestry UNDO i REDO.

*w jakiej
kolejności?*

2PC (2 PHASE COMMITMENT) = DWUFAZOWE ZATWIERDZANIE

Sowa wygłosiła wykład o Niezbędnych Mięśniach Grzbietowych. Już raz zdarzyło jej się mówić o tym Krzysiówi i Puchatkowi. I od tej pory stale wyczekiwała na sposobność zrobienia tego jeszcze raz, ponieważ jest

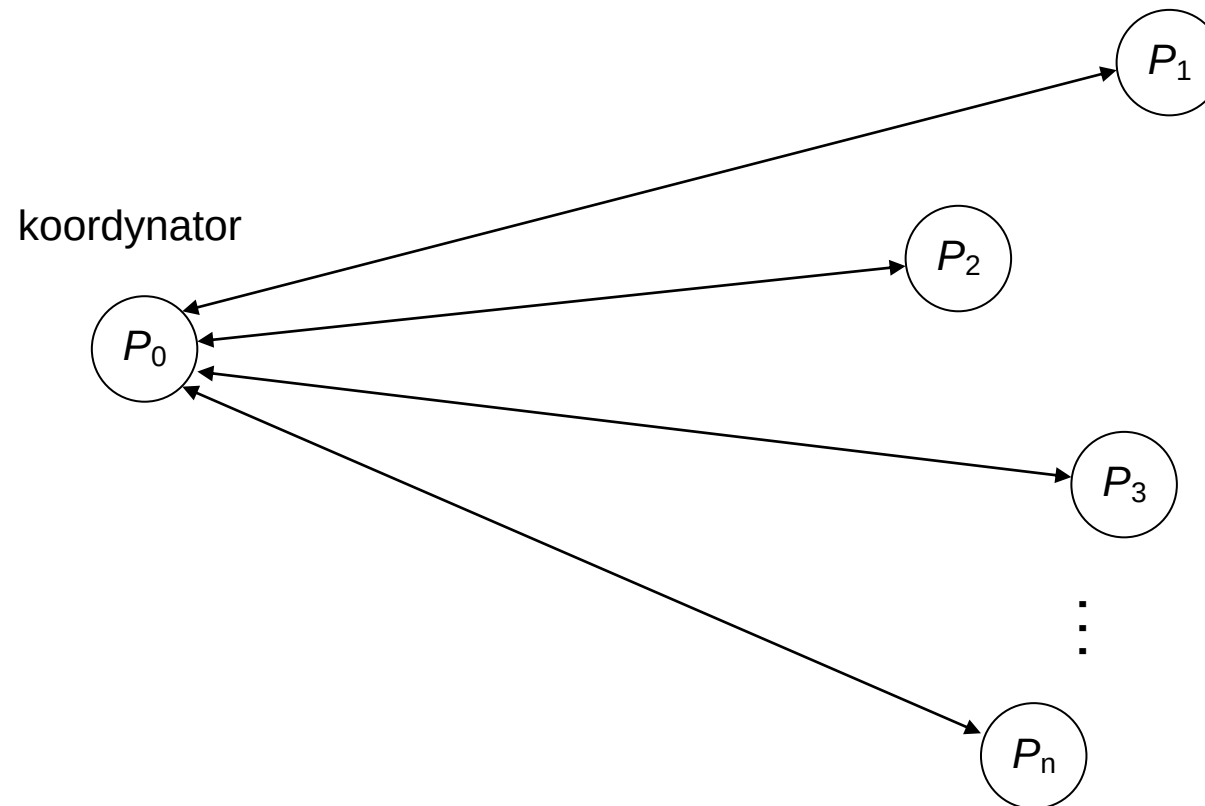


to rzecz, którą można z łatwością wyłożyć dwa razy, zanim ktokolwiek zrozumie, o co chodzi.

Chatka Puchatka



Schemat komunikacyjny





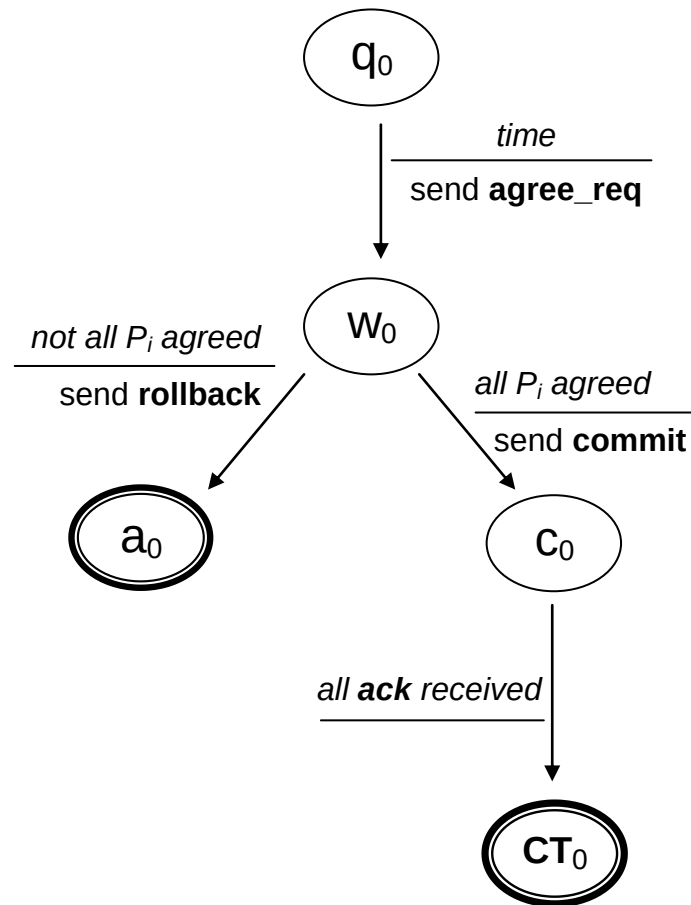
Protokół 2PC (Gray [1])

KOORDYNATOR	UCZESTNIK P_i
Faza 1	
- koordynator wysyła agree_req do wszystkich procesów – uczestników zatwierdzania - koordynator czeka na odpowiedzi od wszystkich procesów	- odebrawszy agree_req proces: ➤ jeśli lokalna część transakcji zakończyła się pomyślnie $\Rightarrow$ zapisuje rejestry UNDO i REDO w pamięci trwałej i odsyła odpowiedź agree ➤ w przeciwnym przypadku $\Rightarrow$ odsyła odpowiedź abort
Faza 2	
- jeśli wszystkie procesy odpowiedziały agree koordynator zapisuje [COMMIT] w rejestrze i wysyła do wszystkich procesów commit; w przeciwnym przypadku wysyła rollback - koordynator czeka na potwierdzenia ack od wszystkich procesów - gdy wszystkie procesy potwierdziły, koordynator zapisuje [COMPLETE]	- odebrawszy commit, proces zwalnia zasoby i blokady i odsyła potwierdzenie ack - odebrawszy rollback, proces wycofuje transakcję, zwalnia zasoby i blokady i odsyła potwierdzenie ack

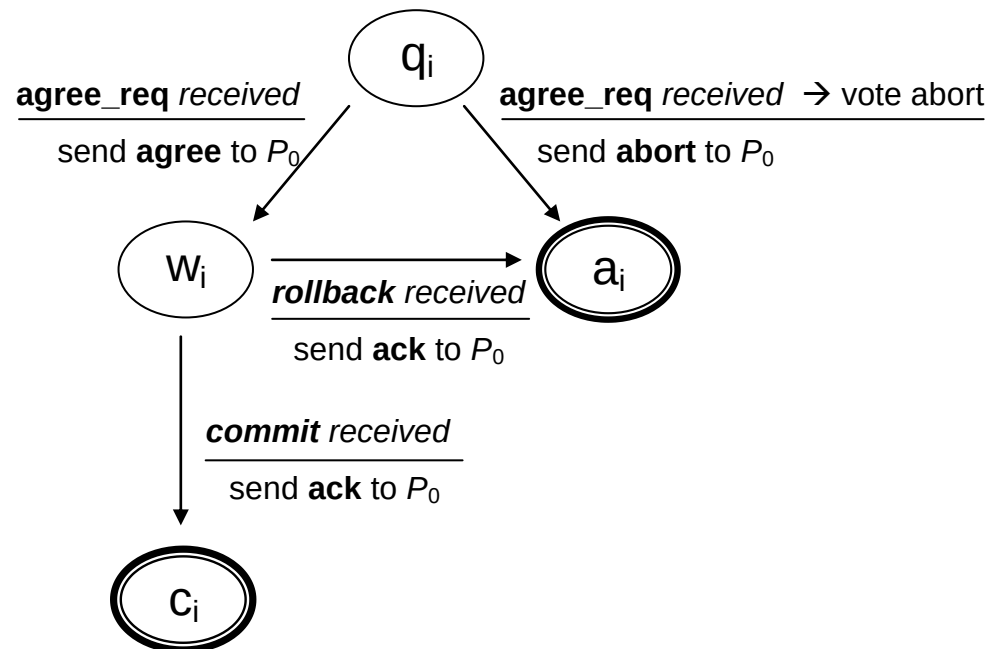


Automaty 2PC

KOORDYNATOR P_0



UCZESTNIK P_i





Awarie

Zgodność: ...

Poprawność: *jeśli wszystkie węzły zakończyły swoje operacje pomyślnie
i wszystkie odpowiedzi węzłów zostały poprawnie dostarczone
transakcja powinna zostać zatwierdzona*

- kanały niezawodne?
- a co z awariami procesów?



Awarie (model *fail-stop*)

KOORDYNATOR	UCZESTNIK P_i
Faza 1	
1. koordynator wysła agree_req do wszystkich procesów 2. koordynator czeka na odpowiedzi od wszystkich procesów K1	1. odebrawszy agree_req proces P1 : ➤ jeśli lokalna część transakcji zakończyła się pomyślnie $\Rightarrow$ zapisuje rejestry UNDO i REDO w pamięci trwałej i odsyła odpowiedź agree ➤ w przeciwnym przypadku $\Rightarrow$ odsyła odpowiedź abort
Faza 2	
1. jeśli wszystkie procesy odpowiedziały agree koordynator zapisuje [COMMIT] w rejestrze i wysyła do wszystkich procesów commit ; w przeciwnym przypadku wysyła rollback 2. koordynator czeka na potwierdzenia ack od wszystkich procesów K2 3. gdy wszystkie procesy potwierdziły, koordynator zapisuje [COMPLETE] K3	1. odebrawszy commit proces P2 zwalnia zasoby i blokady i odsyła potwierdzenie ack 2. odebrawszy rollback , proces wycofuje transakcję, zwalnia zasoby i blokady i odsyła potwierdzenie ack

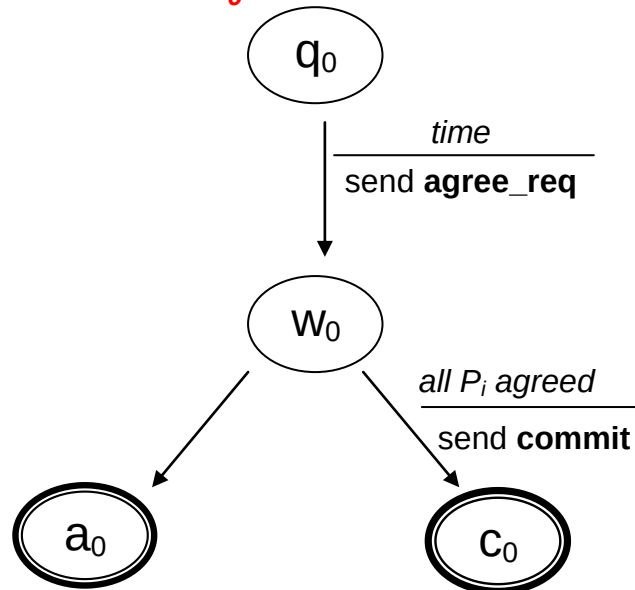


- K1** = koordynator nie zapisał [COMMIT]
recovery: koordynator wysyła **rollback**, UNDO
procesy są zablokowane aż do odebrania **rollback**
- K2** = pomiędzy [COMMIT] a [COMPLETE]
recovery: koordynator wysyła **commit**
procesy są zablokowane aż do odebrania **commit**
- K3** = po [COMPLETE]
recovery: ???
nie ma czego rozważać – pomijamy stan CT_0
- P1** = koordynator nie dostał odpowiedzi na **agree_req**
timeout $\Rightarrow$ koordynator wysyła **rollback**
- P2** = P_i zapisał UNDO i REDO log,
ale koordynator nie dostał odpowiedzi na **ack**
recovery: P_i pyta o ostateczną decyzję koordynatora
jeśli P_i uległ awarii nim zmodyfikował wszystkie obiekty $\Rightarrow$ REDO
alternatywnie koordynator może powtarzać wysyłanie **commit**



Automaty 2PC

KOORDYNATOR P_0



K1 = koordynator nie zapisał [COMMIT]
recovery: koordynator wysyła **rollback**, UNDO
procesy są zablokowane aż do odebrania **rollback**

K2 = pomiędzy [COMMIT] a [COMPLETE]
recovery: koordynator wysyła **commit**
procesy są zablokowane aż do odebrania **commit**

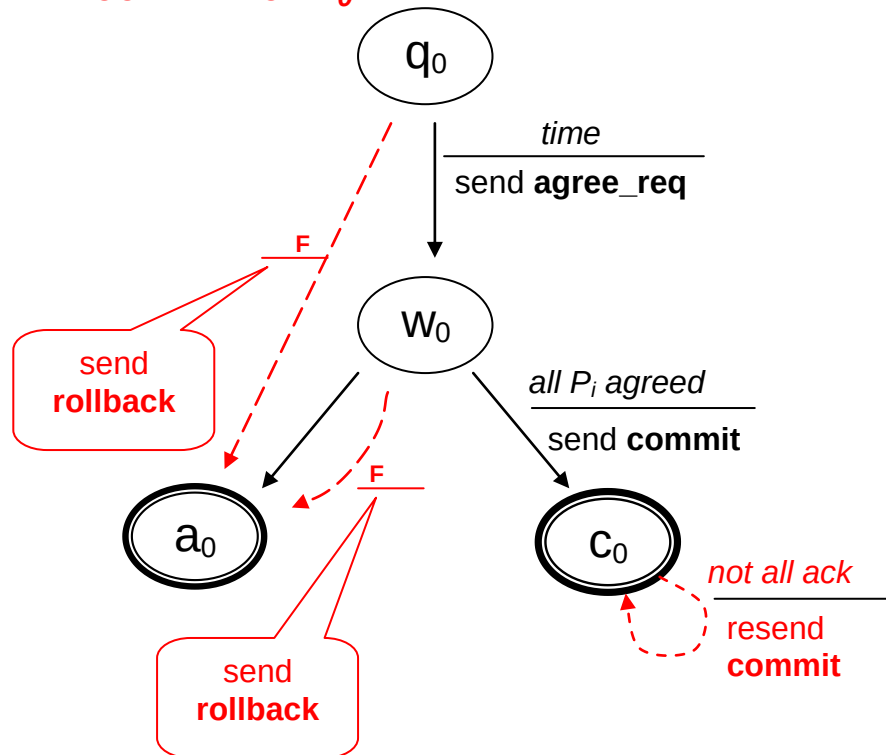
K3 = po [COMPLETE]
recovery: ???
nie ma czego rozważać – pomijamy stan CT_0

P1 = koordynator nie dostał odpowiedzi na **agree_req**
timeout $\Rightarrow$ koordynator wysyła **rollback**



Automaty 2PC z przejściami F i T

KOORDYNATOR P_0



F = failure

K1 = koordynator nie zapisał [COMMIT]
recovery: koordynator wysyła **rollback**, UNDO
 procesy są zablokowane aż do odebrania **rollback**

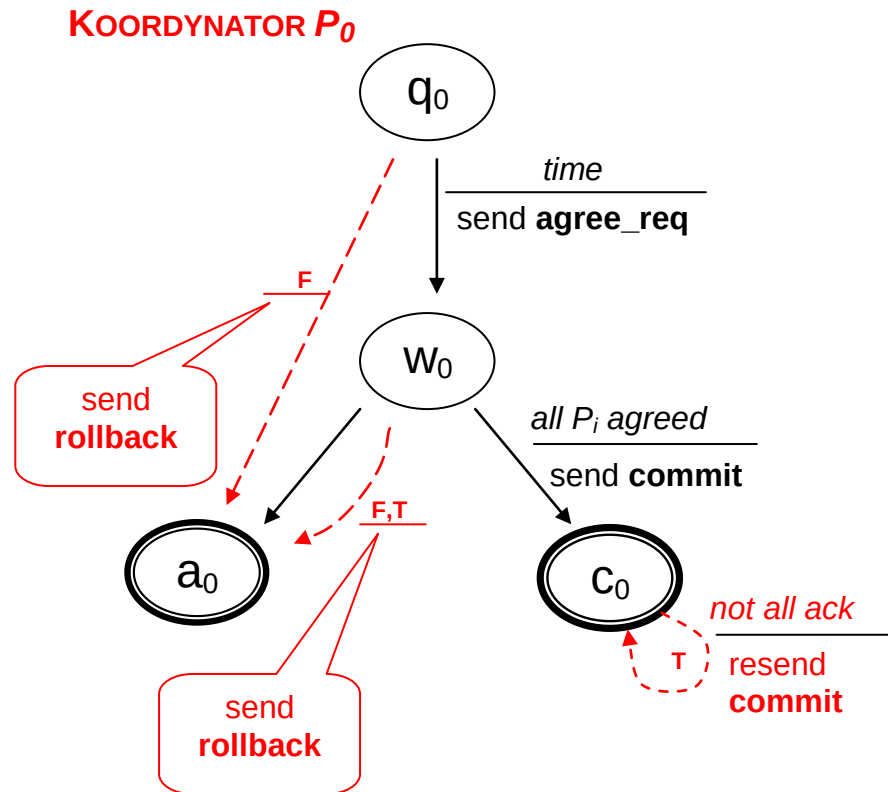
K2 = pomiędzy [COMMIT] a [COMPLETE]
recovery: koordynator wysyła **commit**
 procesy są zablokowane aż do odebrania **commit**

K3 = po [COMPLETE]
recovery: ???
 nie ma czego rozważać – pomijamy stan CT_0

P1 = koordynator nie dostał odpowiedzi na **agree_req**
 timeout $\Rightarrow$ koordynator wysyła **rollback**



Automaty 2PC z przejściami F i T



K1 = koordynator nie zapisał [COMMIT]
recovery: koordynator wysła **rollback**, UNDO
 procesy są zablokowane aż do odebrania **rollback**

K2 = pomiędzy [COMMIT] a [COMPLETE]
recovery: koordynator wysła **commit**
 procesy są zablokowane aż do odebrania **commit**

K3 = po [COMPLETE]
recovery: ???
 nie ma czego rozważać – pomijamy stan CT_0

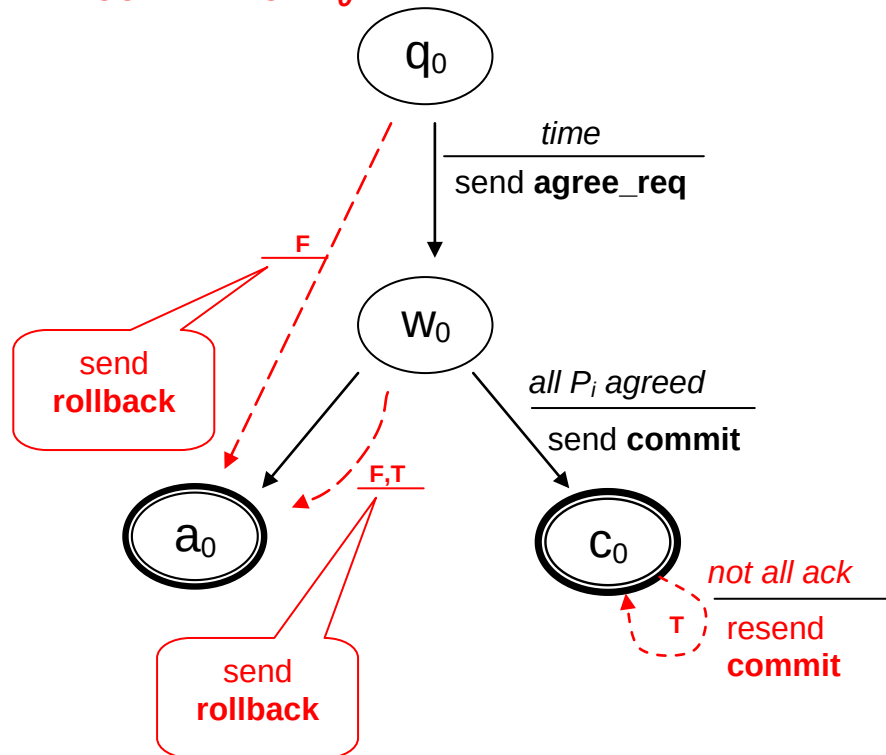
P1 = koordynator nie dostał odpowiedzi na **agree_req**
 timeout $\Rightarrow$ koordynator wysła **rollback**

F = failure T = timeout



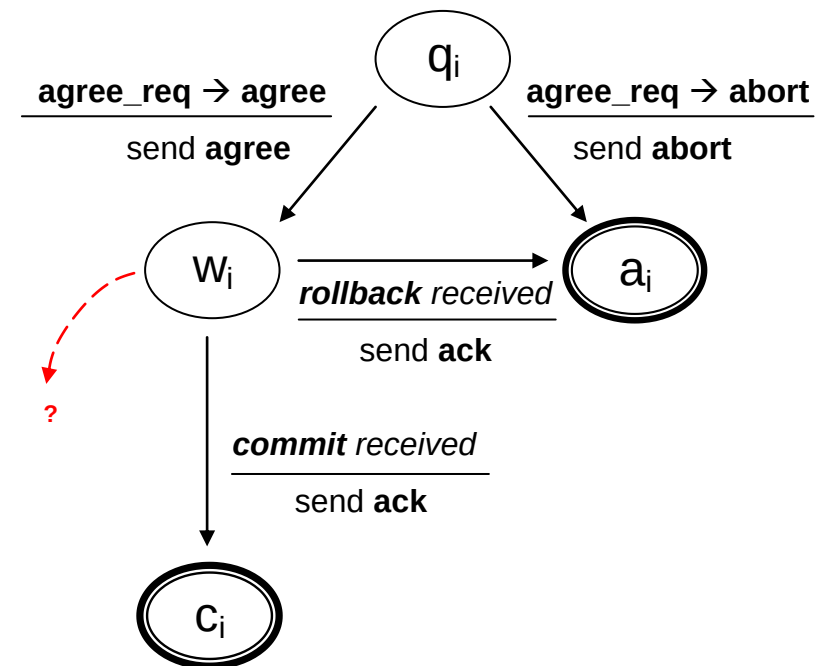
Automaty 2PC z przejściami F i T

KOORDYNATOR P_0



F = failure T = timeout

UCZESTNIK P_i





Zablokowanie przetwarzania

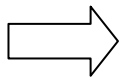
Jeśli koordynator ulegnie awarii, procesy zostają zablokowane aż do jego odtworzenia, przetrzymując zasoby (utrzymując blokady).

Concurrency set:

Niech s_i oznacza stan procesu P_i .

Zbiór wszystkich stanów pozostałych procesów * , które mogą występować współbieżnie ze stanem s_i nazywa się *concurrency set* stanu s_i : $Cset(s_i)$

$$Cset(q_i) = \{ q_0, w_0, q_j, w_j, a_j \}$$

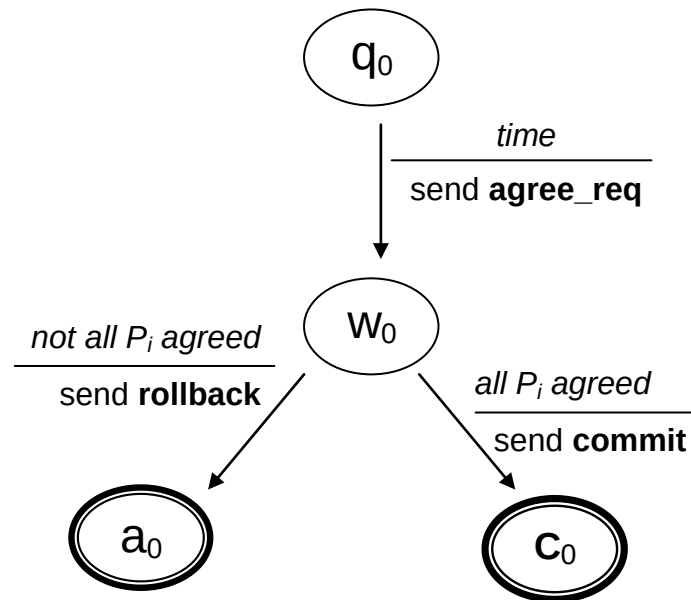


* tu nie uwzględniamy tranzycji F i T $\Rightarrow$

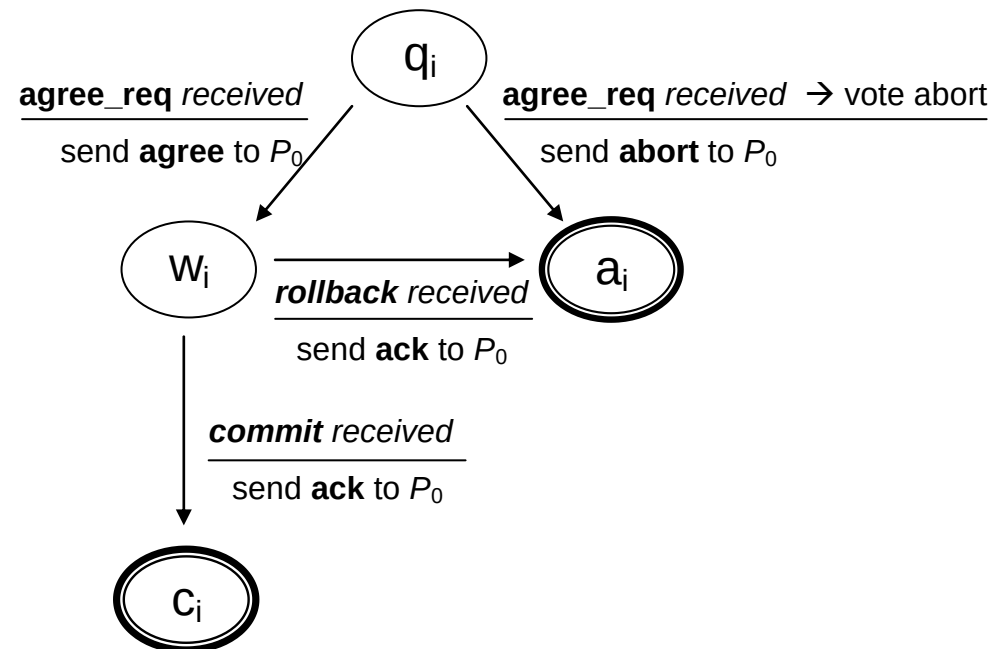


Automaty 2PC: określanie concurrency set

KOORDYNATOR P_0



UCZESTNIK P_i





Zablokowanie przetwarzania

Jeśli koordynator ulegnie awarii, procesy zostają zablokowane aż do jego odtworzenia, przetrzymując zasoby (utrzymując blokady).

Concurrency set:

Niech s_i oznacza stan procesu P_i .

Zbiór wszystkich stanów pozostałych procesów^{*}, które mogą występować współbieżnie ze stanem s_i nazywa się *concurrency set* stanu s_i : $Cset(s_i)$

$$Cset(q_i) = \{ q_0, w_0, q_j, w_j, a_j \}$$

$$Cset(w_i) = \{ w_0, \mathbf{a_0}, \mathbf{c_0}, q_j, w_j, \mathbf{a_j}, \mathbf{c_j} \}$$

Zablokowanie w 2PC może wystąpić, gdy dla dowolnego stanu s $Cset(s)$ zawiera jednocześnie stany $\mathbf{a}$ i $\mathbf{c}$.

* tu nie uwzględniamy tranzycji F i T



3 PC (3 PHASE COMMITMENT)

Algorytm Skeena [2]

Założenia:

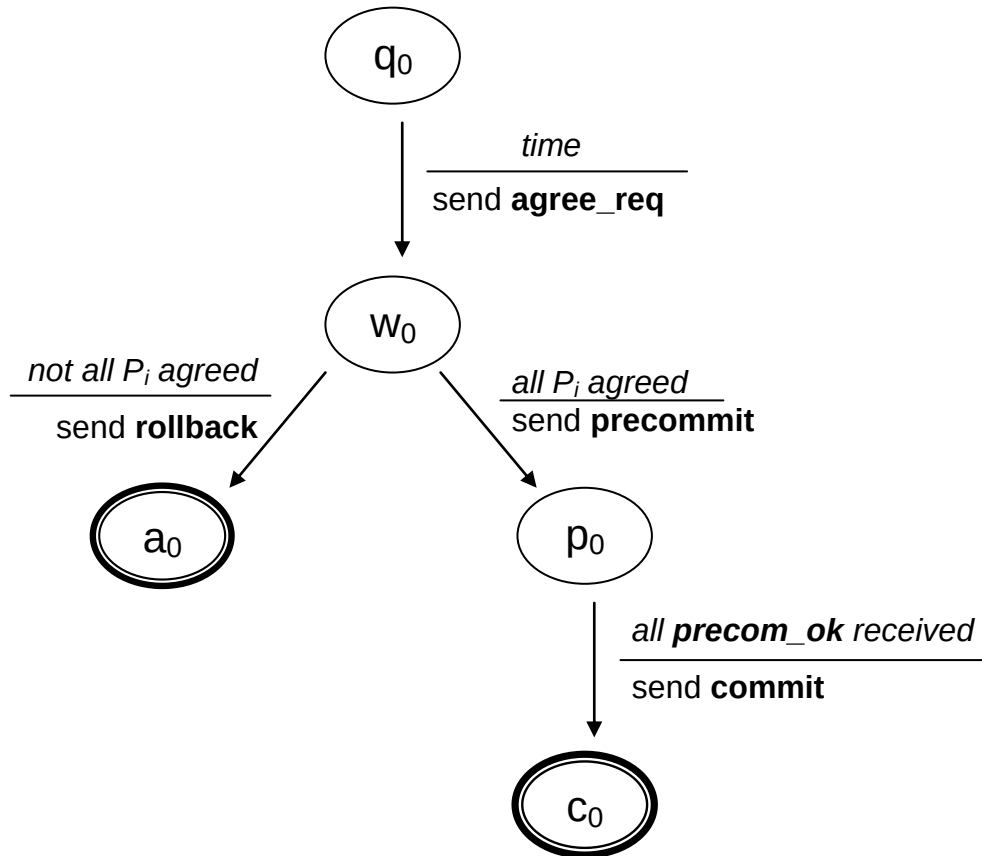
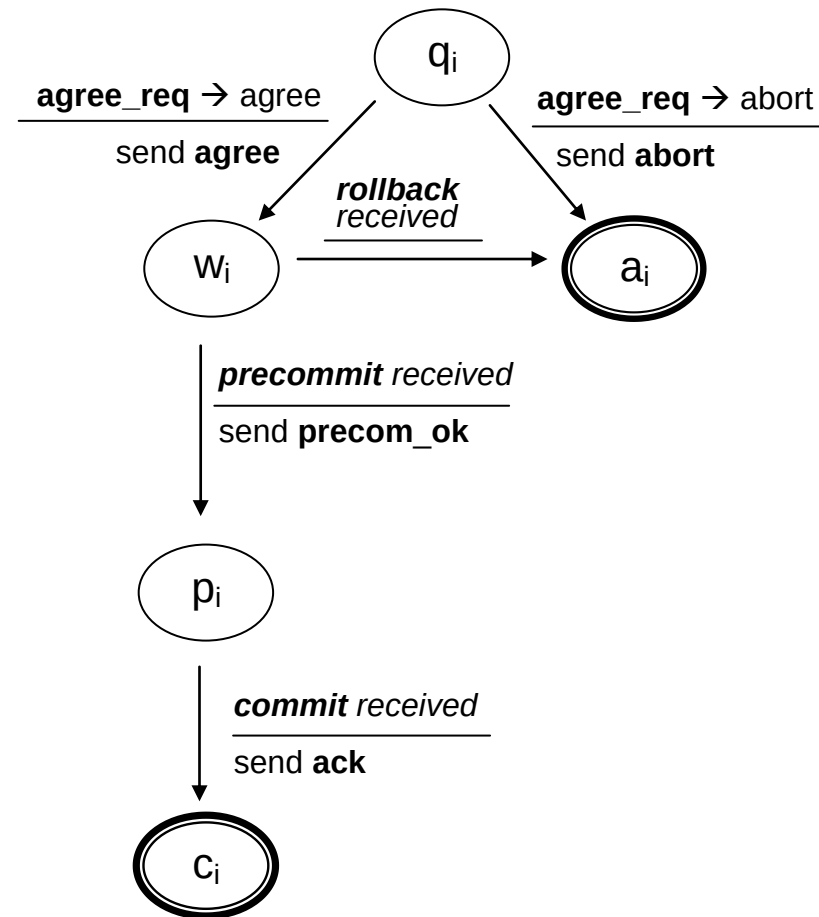
- kanały są niezawodne *uniform reliable reordering channels* (nie gubią komunikatów; kanał dostarczy wysłany komunikat nawet jeśli jego nadawca uległ awarii)
- koordynator używa rozgłaszania *uniform reliable broadcast (URBcast)*
- sieć wykrywa awarię węzła: awarię nieomylnie (!) wskazuje *timeout*
- co najwyżej jeden węzeł ulega awarii

Stan buforowy automatu:

- cel: pozwolić procesom podjąć niezależną decyzję (na podstawie stanu lokalnego)



KOORDYNATOR
...
Faza 2
1. jeśli wszystkie procesy odpowiedziały agree koordynator zapisuje [PRECOMMIT] w rejestrze, wysyła <i>URBcast</i> do wszystkich procesów precommit i czeka na potwierdzenie precom_ok ; w przeciwnym przypadku wysyła rollback
Faza 3
1. gdy wszystkie procesy odpowiedzą precom_ok koordynator zapisuje [COMMIT] w rejestrze i wysyła do wszystkich procesów commit
...

KOORDYNATOR P_0 UCZESTNIK P_i 



Rola stanu p:

$$\text{Cset}(w_0) = \{ q_i, w_i, a_i \}$$
$$\text{Cset}(w_i) = \{ w_0, p_0, a_0, q_j, w_j, a_j, p_j \}$$

...

jeśli $\text{Cset}(s_i)$ zawiera stan c , to wobec awarii koordynatora proces P_i może lokalnie podjąć niezależną decyzję o przejściu do stanu c_i , w przeciwnym przypadku – do stanu a_i .

Zadanie:

- Uzupełnić przejścia automatów 3PC.



Non-blocking property

Algorytm nieblokujący (**non-blocking**) gwarantuje osiągnięcie zakończenia (*termination*) przez co najmniej 1 proces, jeśli co najmniej 1 proces nie ulega awarii.

Wait-freedom property

Wait-freedom to własność gwarantująca, że każdy proces, który nie ulega awarii osiągnie zakończenie przetwarzania (*termination*) bez względu na zachowanie innych procesów.

Własność *wait-freedom* jest silniejsza niż *non-blocking*.

Którą z nich spełnia 2PC? A którą 3PC?



Impossibility Results [3]:

Twierdzenie 1: Nie istnieje nieblokujący protokół rozproszonego atomowego zatwierdzania, który byłby odporny na arbitralne defekty 2 węzłów.

Twierdzenie 2: Nie istnieje nieblokujący protokół rozproszonego atomowego zatwierdzania, który byłby odporny na rozdzielenie sieci (separację na rozłączne podsieci – *partitioning*) przy możliwości gubienia komunikatów.

Twierdzenie 3: Nie istnieje nieblokujący protokół rozproszonego atomowego zatwierdzania, który byłby odporny na wielokrotne rozdzielenie sieci.

→ [4]

LITERATURA

- [1]  J.N. Gray: Notes on Database Operating Systems. "Operating Systems. An Advanced Course", Lecture Notes in Computer Science, vol. 60, Springer, New York, 1979
- [2]  D. Skeen: "Nonblocking Commit Protocols". ACM SIGMOD Conference on Management of Data, 1981
- [3]  D. Skeen, M. Stonebraker: "A Formal Model of Crash Recovery in a Distributed System", IEEE Trans. Software Engineering, vol. 9, no.3, 1983
- [4]  I. Keidar, D. Dolev: "Increasing the Resilience of Atomic Commit, at no Additional Cost". 14th ACM Symposium on Principles of Database Systems, 1995
- [5]  R. Chow, T. Johnson: "Distributed Operating Systems & Algorithms", Addison Wesley Longman, 1997, ch. 12
- [6]  K.P. Birman: "Reliable Distributed Systems", Springer Science, 2005, ch. 14
- [7]  G. Coulouris et al.: "Distributed Systems, Concepts and Design", Addison Wesley, 2012, ch. 17
- [8]  A. Petrov: "Database Internals", Springer Science, 2019, ch. 13